

Samuel Store — E-Commerce Web Application

Project Report

Student: Samuel Bondo

Registration Number: 23527/2023

Institution: UNILAK — Faculty of Computing and Information Sciences

Course: EWA408510 – E-Commerce and Web Application

Academic Year: 2025–2026

Instructor: Eric Maniraguha

1. Introduction

Samuel Store is a full-stack e-commerce web application developed as the final project for the course EWA408510 – E-Commerce and Web Application at UNILAK. The platform enables customers to browse products, manage a shopping cart, place orders, and track purchases online. It also provides a complete administration panel for managing products, orders, customers, inventory, and store settings.

The application is built using modern web technologies — React 19 on the frontend, Node.js with Express on the backend, and MySQL as the relational database. It is fully deployed on cloud platforms and follows professional software development practices including version control, CI/CD pipelines, and Docker containerization.

2. Problem Statement

Many local businesses in Rwanda still operate exclusively through physical stores, limiting their reach to walk-in customers only. Without an online presence, these businesses miss opportunities to serve customers outside their immediate location, operate outside business hours, or scale their sales efficiently.

Additionally, managing inventory, orders, and customer data manually through spreadsheets or paper records is error-prone and time-consuming. There is a clear need for an affordable, modern, and easy-to-use e-commerce platform that allows a local business to sell products online, manage operations from a dashboard, and provide customers with a smooth shopping experience.

3. Project Objectives

The main objectives of this project are:

1. Design and develop a responsive, professional e-commerce web application
 2. Implement full product management with categories, search, and filtering
 3. Build a complete shopping cart and checkout flow with order confirmation
 4. Integrate a MySQL relational database to store all business data
 5. Implement secure user authentication using JWT, bcrypt, and OAuth
 6. Deploy the application online and ensure it remains accessible
 7. Implement a CI/CD pipeline using GitHub Actions for automated builds and testing
 8. Containerize the application using Docker and Docker Compose
 9. Provide an admin dashboard for full store management
 10. Add innovative features including AI-powered chat and real-time notifications
-

4. System Features

Customer-Facing Features

- Responsive homepage with hero carousel and featured products
- Product listing page with search and category filtering
- Product detail pages with images, descriptions, and pricing
- Shopping cart with add, remove, and quantity update controls
- Checkout with saved address picker and manual address entry
- Order confirmation page with order summary
- Order history with tracking steps and HTML invoice download
- User registration and login with JWT authentication
- Google OAuth 2.0 and Facebook OAuth 2.0 sign-in
- OTP-based forgot password (6-digit code via email, 10-minute expiry)
- Customer dashboard with overview, orders, profile, addresses, reviews, notifications, and settings
- AI-powered chat assistant (Google Gemini) for product help
- Privacy Policy page (GDPR and Meta App Review compliant)
- Account data deletion endpoint

Admin Features

- Admin dashboard with real-time statistics
 - Product management (create, update, delete, toggle visibility)
 - Category management
 - Order management with status updates
 - Customer management with account controls
 - Inventory management with stock updates
 - Review moderation with admin replies
 - Banner management for homepage carousel
 - Reports and analytics Store
 - settings management
-

5. Technologies Used

Layer	Technology	Purpose
Frontend	React 19, Vite 8	UI framework and build tool
Routing	React Router 7	Client-side page navigation
HTTP Client	Axios	API requests with JWT interceptor
Backend	Node.js 24, Express.js 4	REST API server
Database	MySQL 8.4	Relational data storage
ORM/Driver	mysql2	Parameterized database queries
Authentication	jsonwebtoken, bcryptjs	JWT tokens and password hashing
OAuth	Passport.js, passport-google-oauth20, passport-facebook	Social login

Email	Nodemailer + Brevo SMTP	OTP password reset emails
File Upload	Multer + Cloudinary	Product image uploads
AI	Google Gemini API	AI chat assistant
DevOps	Docker, Docker Compose	Containerization
CI/CD	GitHub Actions	Automated build and test pipeline
Frontend Hosting	Vercel	Frontend deployment
Backend Hosting	Render	Backend API deployment
Database Hosting	Aiven MySQL 8.4 (Amsterdam)	Cloud database

6. System Architecture

Samuel Store follows a three-tier architecture:

CLIENT TIER

React 19 + Vite — hosted on Vercel
Pages: Home, Products, Cart, Checkout, Dashboard
State: AuthContext, CartContext, SettingsContext
HTTP: Axios with JWT Bearer token interceptor

HTTPS REST API

SERVER TIER

Node.js 24 + Express.js — hosted on Render Routes: /api/products, /api/orders, /api/auth...
Middleware: authenticate, requireAdmin, validate Controllers: authController, orderController...
Models: productModel, userModel, orderModel...

mysql2 connection pool (SSL)

DATABASE TIER

MySQL 8.4 — hosted on Aiven (Amsterdam)
13 tables with foreign key relationships
SSL-encrypted connection in production

Request Flow Example — Place an Order

Customer clicks "Place Order"

React sends POST /api/orders with JWT token

authenticate middleware verifies JWT

validate middleware checks required fields

orderController inserts into orders + order_items tables

cart is cleared

notification created for customer

JSON response returned

React shows OrderConfirmation page

7. Database Design

The database is named `samuel_store` and contains 13 tables.

Core Tables

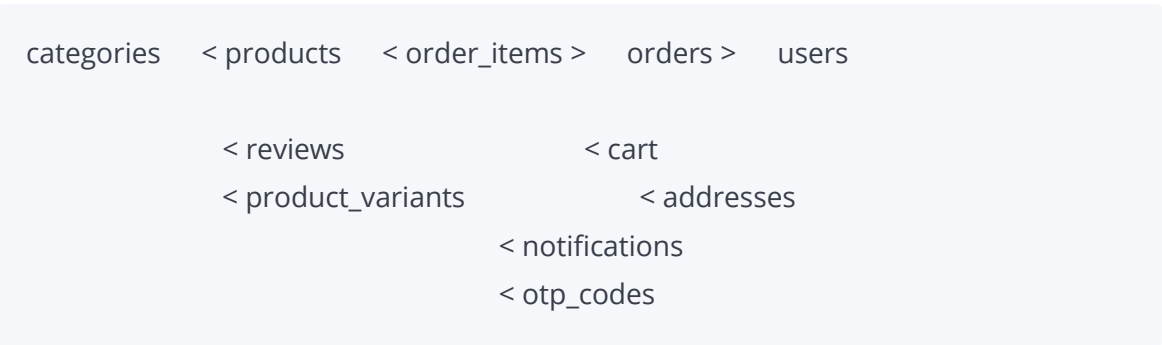
Table	Description	Key Columns
categories	Product categories	id, name
products	Product catalog	id, name, price, stock, image_url, category_id, featured, visible
users	Registered users	id, name, email, password (nullable), role, google_id, facebook_id, auth_provider, avatar
orders	Customer orders	id, user_id, total, status, payment_method, payment_status
order_items	Items within each order	id, order_id, product_id, quantity, price, variant_name
cart	Shopping cart items	id, user_id, product_id, quantity

Supporting Tables

Table	Description
addresses	Up to 5 saved addresses per user
reviews	Product reviews with rating, comment, admin reply

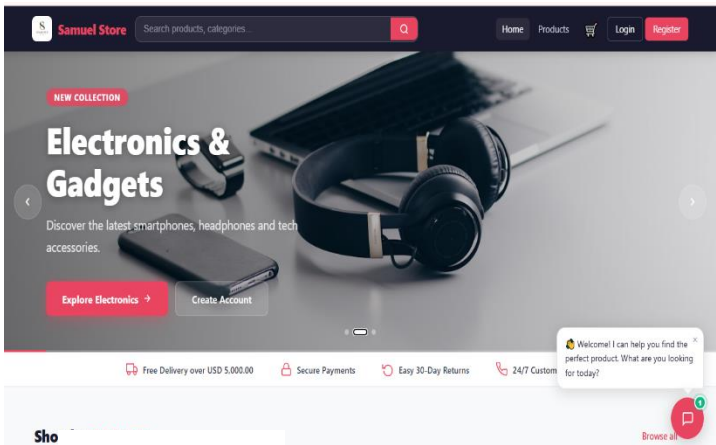
notifications	Per-user notifications with read status
otp_codes	6-digit OTP codes for password reset (10-min expiry)
settings	Store configuration key-value pairs
banners	Homepage carousel banners
product_variants	Product size/colour variants with individual pricing

Entity Relationships

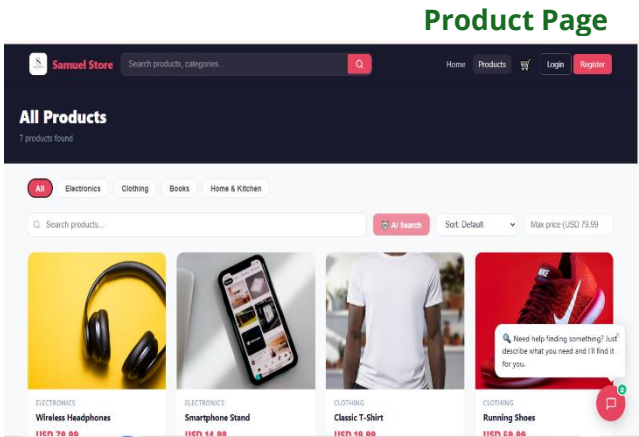


All foreign keys use ON DELETE CASCADE or ON DELETE SET NULL to maintain referential integrity.

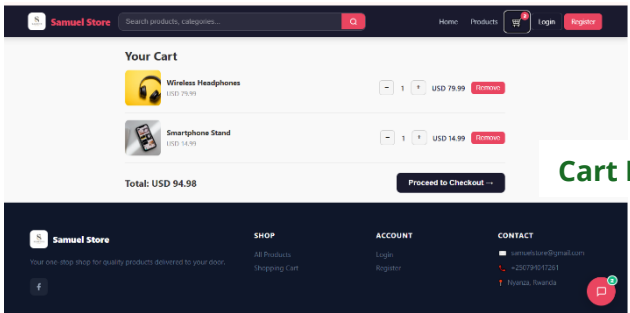
8. Screenshots of the Application



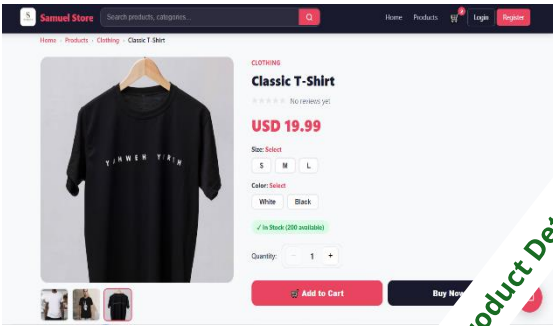
Homepage



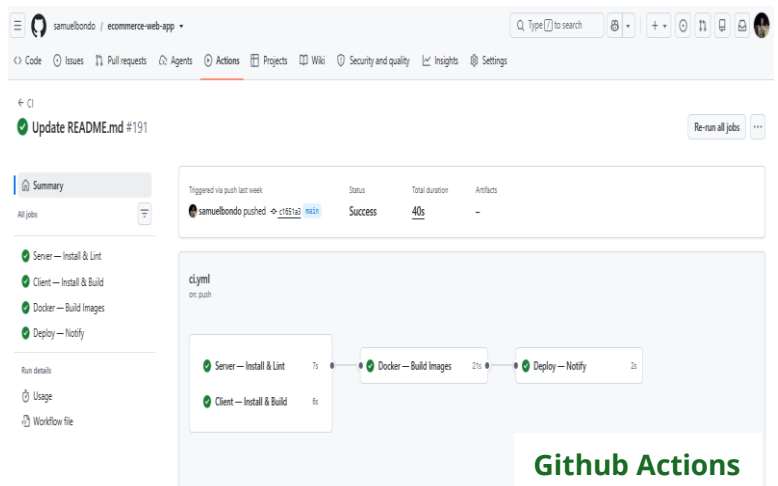
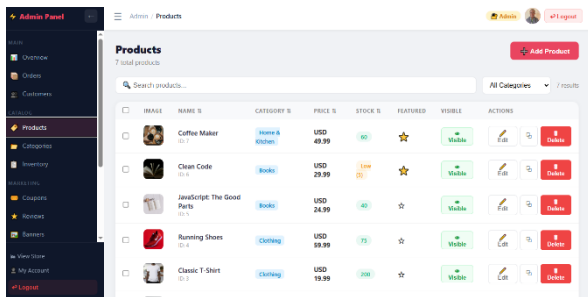
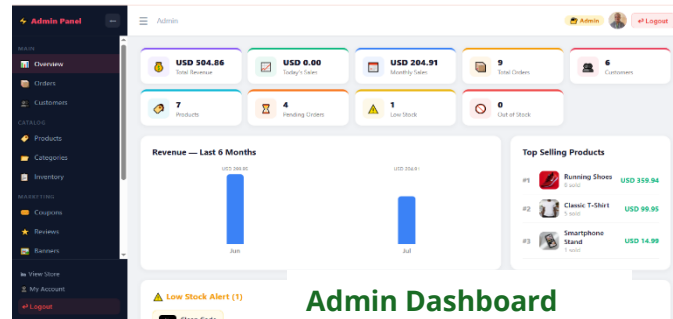
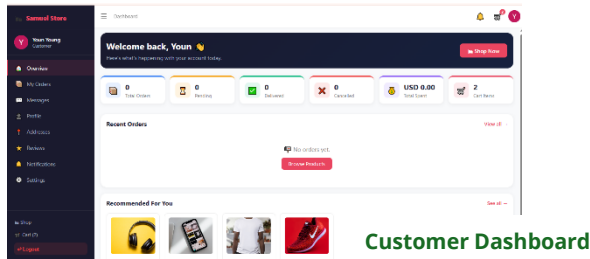
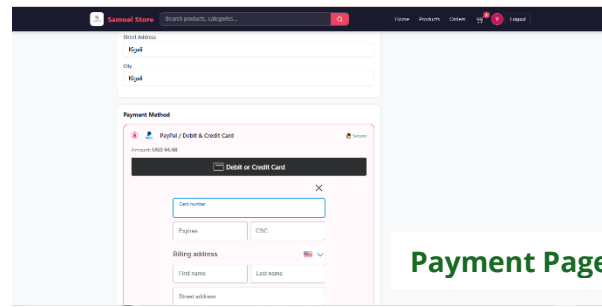
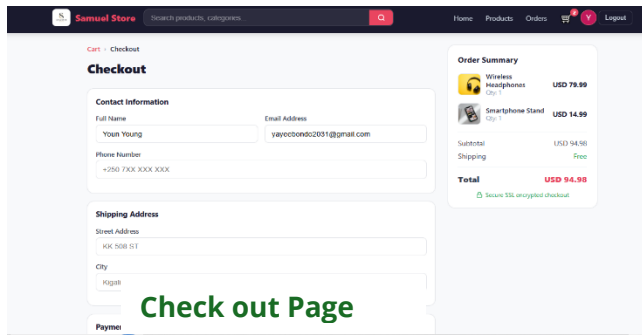
Product Page



Cart Page



Product Details Page



Admin Product-Management

Check the screenshots Folder for more images

URL: <https://github.com/samuelbondo/ecommerce-web-app/actions>

Repository URL: <https://github.com/samuelbondo/ecommerce-web-app>

The repository contains:

Full Source code for both frontend (client/) and backend (server/)

- Database schema and seed files (server/db/)
- Docker configuration (Dockerfile for both services, docker-compose.yml)
- CI/CD pipeline (.github/workflows/ci.yml)
- Complete documentation (README.md)
- Environment variable template (server/.env.example)
- Meaningful commit history reflecting the development process

Deployment

Service	Platform	URL
Frontend	Vercel	https://samuel-store.vercel.app
Backend API	Render	https://samuel-storeserver.onrender.com
Database	Aiven MySQL 8.4 (Amsterdam)	Cloud-hosted, SSL-encrypted

Deployment Process

Frontend (Vercel):

Vercel is connected directly to the GitHub repository. Every push to the main branch triggers an automatic deployment. Vercel runs npm run build (Vite) and serves the compiled static files globally via CDN.

Backend (Render):

Render is connected to the GitHub repository with root directory set to server/. Every push to main triggers an automatic redeploy. The server runs node index.js with all environment variables configured in the Render dashboard.

Database (Aiven):

Aiven provides a managed MySQL 8.4 instance. The backend connects using SSL (rejectUnauthorized: false) via the mysql2 connection pool. The migrate.js script handles schema updates idempotently.

Note: Render free tier spins down after inactivity. The first request after a period of inactivity may take 30–50 seconds. This is normal behaviour on the free plan.

9. CI/CD Implementation

What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Deployment. It is an automated pipeline that runs every time code is pushed to GitHub. Its purpose is to catch errors early, ensure the application builds correctly, and confirm that Docker images can be constructed — all before any broken code reaches the live production environment.

Pipeline Configuration

The pipeline is defined in .github/workflows/ci.yml and is triggered on every push or pull request to the main branch.

Pipeline Stages

git push to main

Job 1: Server — Install & Lint

- Checkout code
- Setup Node.js 24
- npm ci (install dependencies)
- npm test --if-present

- Verify server starts

(must pass)

Job 2: Client — Install & Build

- Checkout code
- Setup Node.js 24
- npm install
- npm run build (Vite compile)

(both must pass)

Job 3: Docker — Build Images

- docker build ./server
- docker build ./client

(must pass)

Job 4: Deploy — Notify

- Print deployment summary
- Vercel + Render auto-deploy

10. Docker Implementation

What is Docker?

Docker is a containerization platform that packages an application together with its entire runtime environment — code, dependencies, configuration — into a portable unit called a container. This eliminates the “works on my machine” problem by ensuring the application runs identically on any system.

Container Architecture

Samuel Store uses Docker Compose to orchestrate three containers:

```
docker-compose up --build
```

Container 1: samuel_store_db

Image: mysql:8.0

Port: 3307

Auto-loads: schema.sql + seed.sql

Health check: mysqladmin ping

```
service_healthy condition
```

Container 2: samuel_store_server

Image: built from server/Dockerfile

Base: node:22-alpine

Port: 5000

Runs: node index.js

```
depends_on server
```

Container 3: samuel_store_client

Image: built from client/Dockerfile

Stage 1: node:22-alpine Vite build

Stage 2: nginx:alpine serve dist

Port: 80

Key Design Decisions

Multi-stage build for the client:

The client Dockerfile uses two stages. Stage 1 uses a Node image to compile the React application with Vite. Stage 2 copies only the compiled output (dist/) into a lightweight Nginx image. This keeps the final image small and production-ready.

Health check with depends_on: condition: service_healthy :

MySQL takes approximately 20–30 seconds to fully initialize. Without the health check condition, the Express server would start, attempt to connect to MySQL before it is ready, and crash. The health check runs mysqladmin ping every 20 seconds and only allows the server container to start once MySQL responds successfully.

Automatic database initialization:

The schema and seed SQL files are mounted into MySQL's docker-entrypoint-initdb.d/ directory. MySQL automatically executes these scripts on first startup, creating all tables and inserting sample data with no manual steps required.

Running the Application with Docker

```
# From the project root directory:  
docker-compose up --build  
  
# Access the application:  
# Frontend: http://localhost  
# API: http://localhost:5000/api/products  
# Database: localhost:3307
```

11. Challenges Encountered

1. CI/CD pipeline failure on client build

The GitHub Actions pipeline was failing on the Client — Install & Build job within 7 seconds. The cause was that npm ci (strict mode) was rejecting the package-lock.json generated on Windows when running on the Linux CI environment. The fix was to replace npm ci with npm install for the client job, which is consistent with what the Dockerfile already uses.

2. Render blocking outbound SMTP

Render's free tier blocks all outbound SMTP connections on standard ports (465, 587 via cPanel/Gmail). This prevented OTP emails from being sent. The solution was to switch to Brevo (formerly Sendinblue), which provides an HTTP-based SMTP relay that works on Render's network. Brevo's free tier allows 300 emails per day, which is sufficient for this project.

3. Google OAuth redirect handling

After Google OAuth authentication, the callback URL redirects the user back to the backend, which then redirects to the frontend with a JWT token in the URL query string. The + characters in JWT tokens were being mangled by URLSearchParams during parsing. The fix was to extract the token using a raw regex match on

```
window.location.search and then apply decodeURIComponent manually.
```

4. MySQL container startup timing in Docker

When running docker-compose up, the Express server was starting before MySQL was fully ready, causing connection errors. The solution was to add a proper health check to the MySQL service and use depends_on: condition: service_healthy in the server service configuration, ensuring the server only starts after MySQL passes its health check.

5. Aiven cloud database SSL requirement

The Aiven MySQL instance requires SSL connections in production. The local XAMPP setup does not use SSL. The db.js configuration was updated to conditionally apply SSL settings only when NODE_ENV=production, allowing the same codebase to work in both local development and cloud production environments.

12. Future Enhancements

1. **Mobile Money Payment Integration** — Integrate MTN Mobile Money and Airtel Money APIs for Rwanda-based payment processing, replacing the current cash-on-delivery method
2. **Progressive Web Application (PWA)** — Add a service worker and web manifest to allow customers to install the app on their phones and use it offline
3. **Real-Time Order Tracking** — Implement WebSocket connections ([Socket.io](https://socket.io)) to push live order status updates to customers without page refresh
4. **Multi-Vendor Marketplace** — Extend the platform to support multiple sellers, each with their own product listings, dashboard, and payout management

5. **Advanced AI Recommendations** — Use purchase history and browsing behaviour to train a recommendation model that suggests relevant products to each customer
 6. **SMS Notifications** — Integrate an SMS gateway (e.g., Africa's Talking) to send order confirmations and delivery updates via SMS to Rwandan phone numbers
 7. **Product Reviews with Images** — Allow customers to upload photos with their reviews for more authentic social proof
 8. **Coupon and Discount System** — Complete the coupon management module already scaffolded in the admin dashboard with full redemption logic at checkout
-

13. Conclusion

Samuel Store is a complete, production-grade e-commerce web application that successfully addresses all requirements of the EWA408510 final project. The platform provides a smooth shopping experience for customers and a powerful management interface for administrators.

The project demonstrates practical application of full-stack web development skills — from designing a normalized relational database and building a secure REST API, to developing a responsive React frontend and deploying the entire system to cloud infrastructure.

Beyond the core requirements, the project implements several advanced features including Google and Facebook OAuth, OTP-based password reset, an AI chat assistant powered by Google Gemini, real-time notifications, and a comprehensive admin analytics dashboard. These additions reflect a commitment to building software that is not only functional but also modern, secure, and user-focused.

The DevOps implementation — with a 4-stage GitHub Actions CI/CD pipeline and a 3container Docker Compose setup — demonstrates an understanding of professional software delivery practices. Every push to the main branch automatically validates the server, builds the React application, constructs Docker images, and confirms deployment readiness.

This project has been a valuable learning experience in building, deploying, and maintaining a real-world web application from concept to production.